arXiv:submit/2504425 [cs.AI] 11 Dec 2018

Reading Industrial Inspection Sheets by Inferring Visual Relations

Rohit Rahul[1], Arindam Chowdhury[1], Animesh[1], Samarth Mittal[2], and Lovekesh Vig[1]

[1] TCS Research, Delhi, India
[2] BITS Pilani, Goa Campus, India
rohit.rahul@tcs.com

Abstract. The traditional mode of recording faults in heavy factory equipment has been via handmarked inspection sheets, wherein a machine engineer manually marks the faulty machine regions on a paper outline of the machine. Over the years, millions of such inspection sheets have been recorded and the data within these sheets has remained inaccessible. However, with industries going digital and waking up to the potential value of fault data for machine health monitoring, there is an increased impetus towards digitization of these handmarked inspection records. To target this digitization, we propose a novel visual pipeline combining state of the art deep learning models, with domain knowledge and low level vision techniques, followed by inference of visual relationships. Our framework is robust to the presence of both static and non-static background in the document, variability in the machine template diagrams, unstructured shape of graphical objects to be identified and variability in the strokes of handwritten text. The proposed pipeline incorporates a capsule and spatial transformer network based classifier for accurate text reading, and a customized CTPN [15] network for text detection in addition to hybrid techniques for arrow detection and dialogue cloud removal. We have tested our approach on a real world dataset of 50 inspection sheets for large containers and boilers. The results are visually appealing and the pipeline achieved an accuracy of **87.1%** for text detection and **94.6%** for text reading.

1 Introduction

Industrial inspection of factory equipment is a common process in factory settings, involving inspection engineers conducting a physical examination of the equipment and subsequently marking faults on paper based inspection sheets. While many industries have digitized the inspection process [1][5][11], paper based inspection is still widely practiced, frequently followed by a digital scanning process. These paper based scans have data pertaining to millions of faults detected over several decades of inspections. Given the tremendous value of fault data for predictive maintainence, industries are keen to tap into the vast reservoir of fault data stored in the form of highly unstructured scanned inspection sheets and generate structured reports from them.

However, there are several challenges associated with digitizing these reports ranging from image preprocessing and layout analysis to word and graphic item recognition [10]. There has been plenty of work in document digitization in general but very little prior work on digitization of inspection documents. In this paper, we have addressed the problem of information extraction from boiler and container inspection documents. The target document, as shown in Figure 1, has multiple types of printed machine line diagrams, where each diagram is split into multiple zones corresponding to different components of the machine. The inspection engineer marks handwritten damage codes and comments against each component of the machine (machine zone). These comments are connected via a line or an arrow to a particular zone. Thus, the arrow acts as a connector that establishes the relationship between a text cloud containing fault codes, and a machine zone.

2 Problem Description

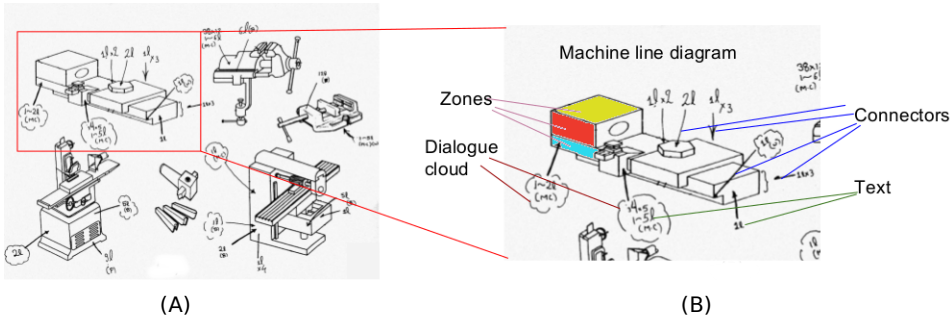

Fig. 1. (A) Overview of an inspection sheet (B) Essential components of the sheet.

In this work, we strive to extract relevant information from industrial inspection sheets which contain multiple 3D orthogonal machine diagrams. Figure 1(A) shows one such inspection sheet consisting of 7 machine diagrams. We define a *set* as a collection of inspection sheets which contain identical machine diagrams while the individual machine diagrams in an inspection sheet are called *templates*, as shown in Figure 4. Each template consists of multiple zones. In Figure 1(B) we mark individual zones with different colors. In industrial setting, an inspector goes around examining each machine. If he detects any damage in a machine then he identifies the zone where the damage has occured. He then draws an entity which we call as *connector* as shown in Figure 1(B) and writes a damage code at the tail of the connector. Each code corresponds to one of the predefined damages that could occur in the machine. This damage code is shown

as text in Figure 1(B). Often, the text is enclosed in a bubble or a cloud structure that carry no additional information of relevance but adds to the overall complexity of the problem. Our task is to localize and read the damage codes that are written on the inspection sheet and associate each damage code with the zone against which it is marked and store the information in a digital document. This allows firms to analyze data on their machines, that was collected over the years, with minimum efforts.

3 Proposed Method

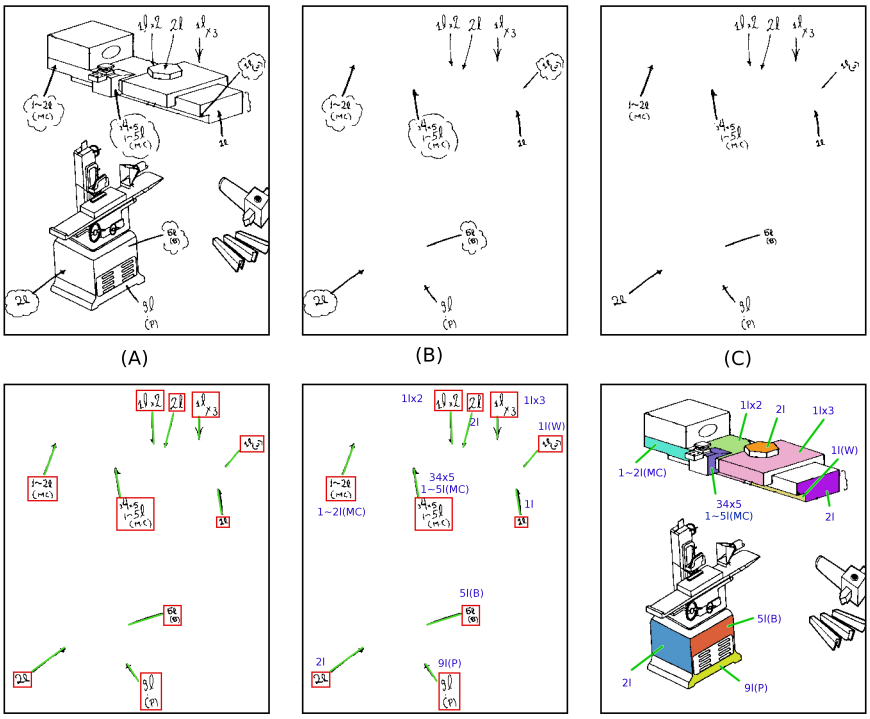

Fig. 2. Overview of the framework: (A) Original Input Image (B) Template Removal (C) Cloud Removal (D) Text and Arrow Localization (E) Text Reading (F) Text-to-Zone Association

We propose a novel framework for extracting damage codes, handwritten by a user on an inspection sheet and then associating the same with correponding zones, as shown in Figure 2. The major components of our model are described in detail in this section. We first remove the templates and the clouds. Then, we localize the text patches and the connectors. Further, we combine the information on the connectors and text patches for more accurate localization and

extraction of the text. This is followed by reading of the damage codes. Finally, we associate the damage codes with the zones, leaveraging the knowledge about the connectors. This process successfully establishes a one-to-one mapping between the zones and corresponding damage codes.

3.1 Template Extraction and Removal

An inspection sheet is essentially composed of a static and a dynamic part. The static part is the 3D orthogonal view of a machine that remains constant over a set of inspection sheets. On the other hand, the dynamic part consists of arrows, clouds and text that is added by the user on top of the static part, as shown in Figure 3. Our goal is to find specific components of the dynamic part and to identify relationships among those parts. We have found that at times static part interferes with the detection of the dynamic part and therefore, as a first step, we remove the static part from the input images.

Template Extraction : Having established the presence of static and dynamic parts in a particular set of sheets, we automate the process of extracting the templates in the sheet. The process involves invertion of the images followed by depthwise averaging and a final step of adaptive thresholding. This generates an image containing just the template. We have noticed that though there are multiple sheets with similar templates, the relative start point of each template is not consistent among the sheets. Hence there is a need to find the individual templates and localize them in the input image. To this end, we find contours on the depth averaged image and then arrange all the detected contours in a tree structure with the page being the root node. In such an arrangement, all nodes at depth 1 are the templates.

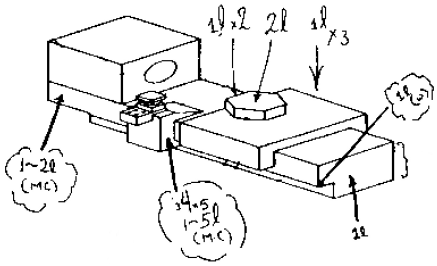

Fig. 3. Original Image

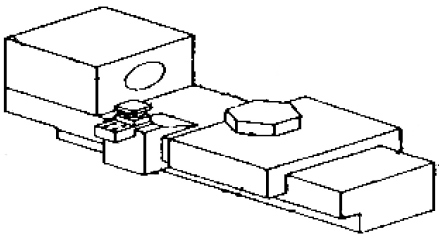

Fig. 4. Template in the image

Template Localization : Now that we have the templates, we use *Normalized Cross Correlation* [16] to match the templates with input sheets. This gives us the correlation at each point in the image. By taking the point exhibiting

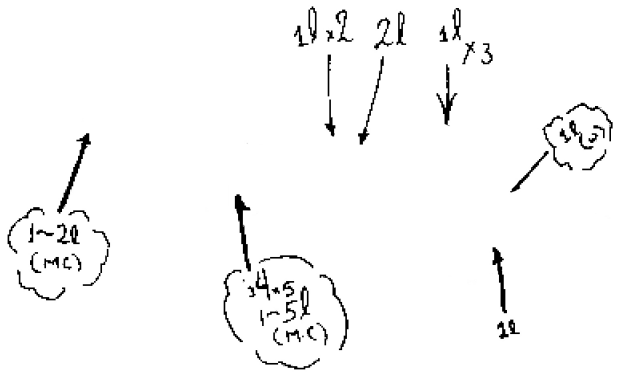

Fig. 5. Image with template removed

maximum correlation, we can find the location of the template present in the image.

$$R(x,y) = \frac{\sum_{x',y'}(T(x',y') * I(x+x',y+y'))}{\sqrt{\sum_{x',y'} T(x',y')^2 * \sum_{x',y'} I(x+x',y+y')^2}}$$

Template Subtraction : To remove the template that was localized in the previous step we use the operator Not(T(i, j)) and R(i, j) on two images T and R, where T is the template image and R is the input image. The resulting image after template subtraction is shown in Figure 5.

3.2 Dialogue cloud segmentation and Removal

Dialogue cloud contains the text/comment in documents as shown in Figure 6. They are present sporadically in the inspection sheet and interfere with the detection of dynamic parts like connectors and text. We have used the encoder-decoder based *SegNet* [2] architecture for segmenting out dialouge clouds. It was trained on a dataset of 200 cloud images to distinguish 3 classes namely background, boundary and cloud. Generally, it was able to learn the structure of the cloud. At times, the segnet would classify a few pixels as background which would lead to introduction of salt and pepper noise around the place where the cloud was present, but we address this issue while text reading by performing median filtering.

3.3 Connector Detection and Classification

Connectors established a one-to-one relationship between text and its corresponding zone. They sometimes manifest as arrows with a prominent head but often they are just lines or multiple broken pieces of a line, as shown in the image, making the automation process far more complex. We tackle this problem

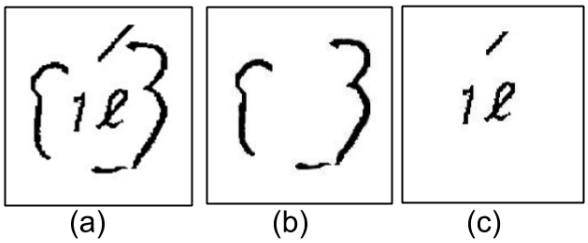

Fig. 6. Cloud segmentation a. input image b. mask generated by segnet c. mask removed

using two approaches :

1. CNN to detect the arrows with prominent heads
2. Detection of Lines

Arrow Classification : As the first step we extract all the connected components from the image to send them to our classifier. We train the *Convolutional Neural Network* (CNN) on 2 classes which are Arrow and background. We modified the architecture of *Zeiler-Fergus* Network (ZF) [17] and show that our network outperforms ZF network in the task of arrow classification by a considerable margin. We trained the classifier to learn the features of the connectors which have a prominent arrow like head. We observed that including the connectors which do not have a prominent head (i.e they are just a line) confuses all CNN models and the precision falls dramatically. To detect arrows in the input image, we feed each connected component found after the template removal to the CNN classifier. All the connected components that are arrows and have a prominent head are classified as such, subsequently, we use the information of the text patches to find out the head and tail point of the arrow.

Line Detection : Most of the arrows having a prominent head would be, at this point, detected by the arrow CNN. Here, we describe the process of detecting arrows that have been drawn as a line (i.e. without a prominent head). To this end, we use a three-step approach. The first step involves detection of various lines that were present in the input image after the removal of templates through

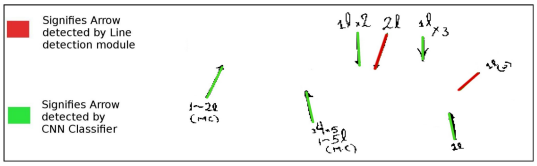

Fig. 7. Arrow detection

hough lines. This is followed by line merging and line filtering where we filter the lines based on the association with the text patch. The filtering step is required because a lot of noise would also be detected as lines which can be filtered leveraging the knowledge gained after text patch detection and association. We further elaborate on the filtering step in Section 3.5.

Line Merging : As can be seen in Figure 5 that after template removal a lot of arrows are broken into segments and hence for each segment a seperate line would be detected. As a result there would be multiple lines for a single arrow. Therefore we merge the lines if they have the same slope and the euclidean distance between them is within 50 px. The resulting image after arrow classification and line detection and merging is shown in Figure 7.

3.4 Text Patch Detection

The text patches in the input image is usually present in the vicinity of a template. To detect these text patches, we employ *Connectionist Text Proposal Network* (CTPN) [15] which has proven to be quite effective in localizing text lines in scene images. With a bit of fine-tuning the same model was able to locate text boxes in the input images. Initially, we trained the CTPN on full size images but it failed to produce desired results. It captured multiple text patches, that occur colinearly, in a single box. This anomaly resulted from the low visual resolution of the individual text patches when looked at from a global context which is the entire image. The network simply captured any relevant text as a single item if they are horizontally close. As a resolution of the same, we sample 480x360 px windows from the input image with overlap. These windows offer better visual seperation between two colinear text patches, resulting in superior localization. Nevertheless, not all text boxes that contained more than one text patch can not be eliminated by the same, as shown in Figure 8(A).

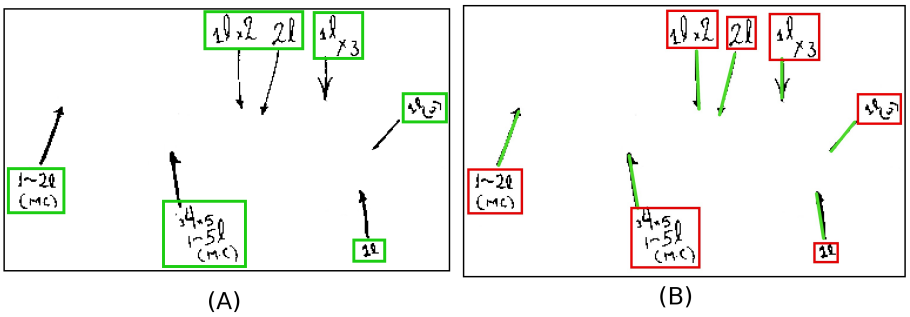

Fig. 8. (A) Text boxes detected by CTPN (B) Final text boxes after associaton with connectors

3.5 Connector Filtering and Text Patch Association

For complete resolution of the problem discussed in last section, we use the available information from the detected arrows, as each text patch must have a corresponding arrow tail pointing to it. We associate each arrow to one of the bounding boxes by extrapolating the arrow tails. Once all the detected arrows are associated to a bounding box, we cluster the text patches present, with the number of clusters being equal to the number of arrows associated to that bounding box. This means that if there exists a bounding box that has two or more arrows associated to it, we will obtain the same number of text patches as the number of arrows. We use K-means clustering for this purpose, where K is the number of arrows associated to a CTPN bounding box. This ensures that there will always be only one text patch associated to a single arrow, as shown in Figure 8(B). Once we have the bounding boxes of the text patches, we extract them and send them to the reading pipeline.

3.6 Text Reading

This section describes the text reading component of our model. Input to this system is a set of text patches extracted from the inspection sheets. Each patch contains handwritten alpha-numeric codes corresponding to a particular kind of physical damage. Major challenges arise from the fact that these damage codes are not always structured horizontally in a straight line but consist of multiple lines with non-uniform alignments, depending on the space available to write on the inspection sheets, as shown in Figure 9. Moreover, the orientation of the characters in these codes are often irregular making the task of reading them even more difficult.
Due to these irregularities, it was difficult to read an entire text sequence as a whole. Instead, we designed our model to recognize one character at a time and then arrange them in proper order to generate the final sequence. The model consists of a segmentation module that generates a set of symbols from the parent image in no particular order, followed by a ranking mechanism to arrange them in standard human readable form. We then employ two deep neural networks to recognize the characters in the sequence. The final component is a correction module that exploits the underlying syntax of the codes to rectify any character level mistake in the sequence.

Segmentation of individual characters in the image patch is performed using *Connected Component Analysis*(CCA). As CCA uses a region growing approach, it can only segment out characters that neither overlap nor have any boundary pixels in common. So, the CCA output may have one or more than one characters in a segment. In our experiments, we found that the segments had a maximum of two characters in them.

Ranking of segmented characters is described in Algorithm 1. It takes a list of unordered segments and returns another that has the characters arranged in a

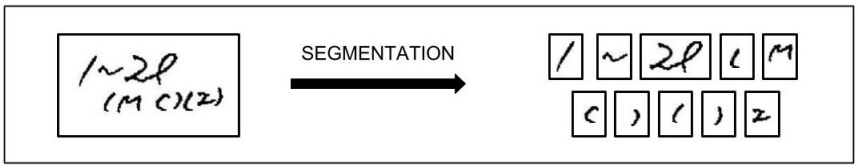

Fig. 9. Segmentation output of text patch

human readable form i.e. left-to-right & top-to-bottom, as shown in Figure 10.

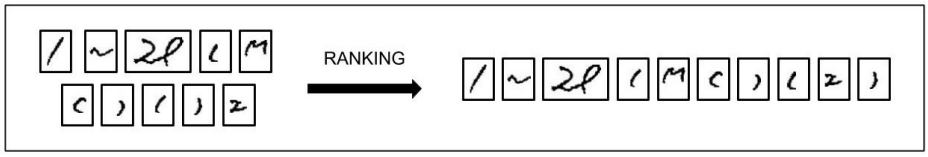

Fig. 10. Segmented output of text patch ranked in human readable format

Character Recognition is implemented as a two-step process. First step is to determine whether a segment contains one or two characters. Towards this end, we use *Capsule Network* (CapsNet) [12] which performs remarkably well in classifying multiple characters with considerable overlap. We modified the standard formulation of CapsNet by introducing a new output class, *None* representing the absence of any character in the image. Therefore, in case there is only a single character present in the segment, CapsNet predicts *None* as one of the two classes. In spite of being a powerful classification model, the performance of CapsNet on the test data was limited. This necessitated the second step in which we use a *Spatial Transformer Network* (STN) [7] to recognize single character segments. STN consists of a differentiable module that can be inserted anywhere in CNN architecture to increase its geometric invariance. As a result, STN proved to be more effective in addressing randomness in the spatial orientation of characters in the images, thereby boosting the recognition performance. Finally, in case of segments that had two characters, we take the CapsNet predictions as the output as STN cannot classify overlapping characters. This scheme is described in Figure 11.

Correction module incorporates domain knowledge to augment neural network predictions. It has two parts. First, a *rule-based system* that uses the grammar of the damage codes to rectify predictions of the networks. For example, as per the grammar, an upper case "B" can only be present between a pair of parenthesis, i.e. "(B)". If the networks predict "1B)", then our correction module would correct this part of the sequence by replacing the "1" by a "(". On top of it is

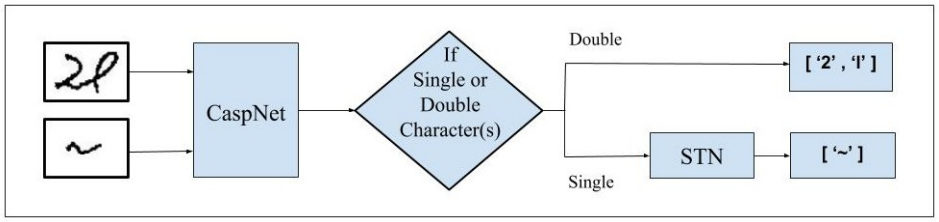

Fig. 11. Capsnet and STN

an *edit-distance based* module which finds the closest sequence to the predicted damage sequence from an exhaustive list of possible damage codes. An example is shown in Figure 12.

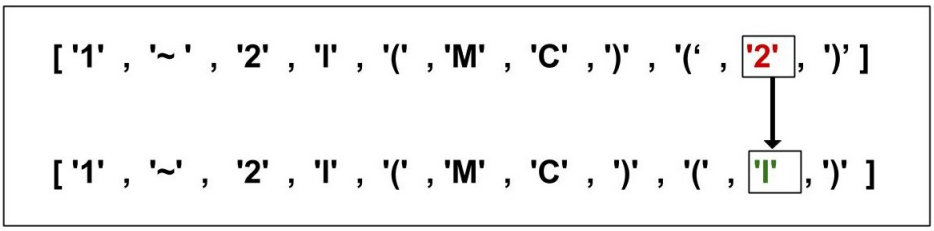

Fig. 12. Correction made in the CapsNet and STN predictions based on the grammer of the defect codes

3.7 Zone mapping

After getting the location of the arrows and associating them with the corresponding text, we now have to map the damage codes to the zone. A sample machine part with different zones are shown in Figure 13. Here, arrows are used to describe this relationship as the head of an arrow points to the relevent zone and the tail of the same points to the text patch that is to be associated with the corresponding zone. We have already done the latter in the previous section and now we are left with the task of finding out the relevent zone to which the arrow is pointing. We observed that this problem can be easily solved by ray casting algorithm [14]. If we extend a ray from the head of the arrow, the zone that it intersects first is the relevent zone and can be mapped to the text patch.

We summerize the proposed method in the following flow diagram :

Algorithm 1 Ranking

Require: $seg_list[s_1, s_2, s_3, ..., s_n]$: Segment $s_i = [(x_1 : x_{m_i}), (y_1 : y_{m_i})]$, a collection of pixel coordinates.

procedure RANKING(seg_list)
  using s_i, calculate extremes and centroid of segmented images, $y_{i_top}, y_{i_bottom}, x_{i_left}, x_{i_right}, y_{i_cen}$ and the x_{i_cen}
  sort seg_list in ascending order of y_{i_cen}parts
  $line_bottom \leftarrow y_{i_top}$; $line_begin_Idx \leftarrow 0$
  for i in (1,len(seg_list)) **do**
    if $y_{i_top} >$ overlap_thresh **then**
      sort previous line based on x_{i_left} ; $line_begin_Idx \leftarrow i$
    line_begin_Idx = max(y_{i_top} , line_bottom)
  sort last line based on x_{i_left}
  return seg_list

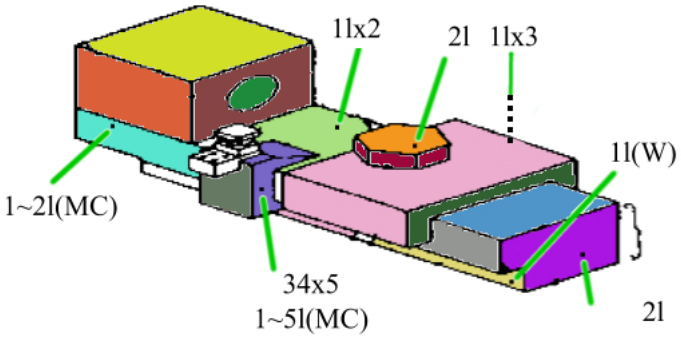

Fig. 13. In this example , different zones of a subassembly are shown with different colors and the rays are shown as dotted red lines extending from the arrow head

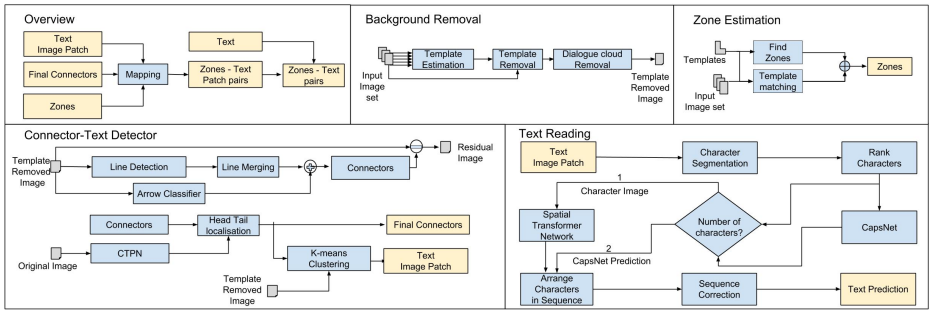

Fig. 14. Flow diagram of the proposed framework. Each component represents a set of operations that extract a particular information from the input images.

4 Experiments

4.1 Implementation Details

We have a confidential dataset provided by a firm. It has 72 different kinds of machine structures distributed across 10 sets of images. There were 50 equally distributed images for testing. This implies that a particular set has same machine line diagrams forming the static background. For training purpose, a separate set of 450 images are kept with same distribution of background machine line diagram sets. All the sheets are in JPEG format with resolution of 3500×2400 sq. px. They have been converted into inverted binarized version where the foreground is white and background is black. The conversion is done by Otsu's binarisation.

Dialogue Cloud Segmentation: For this process, we have used the SegNet [2] model to train on 200 images. Two classes are cloud pixels and background. As there is an imbalance, the classes are weighted by 8.72 for the foreground and 0.13 for the background.

Arrow Classifier: The classifier is inspired from [13]. It includes 6 convolution layers and 2 fully connected layer with ReLU [9] activation. Max pool and dropout (with 0.5 probability) were used for regularization. We set the learning rate of 0.001 and used the Adam [8] optimizer with cross entropy loss to train it on 800 images with equal number of images per class. We initialized the network using Xavier initializer [4] and trained the model till best validation accuracy achieved after 50 epochs. We used *Batch Normalization* [6] with every convolution layer so as to make the network converge faster. The network is 99.7% accurate on a balanced test set of 400 images. The input images are resized to (128×128) with padding such that the aspect ratio of the images is undisturbed.

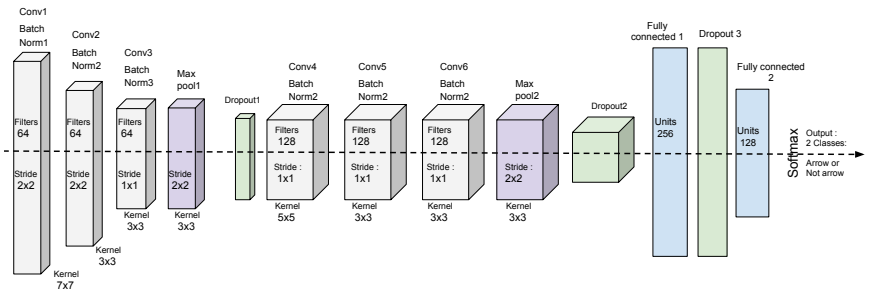

Fig. 15. Architecture of CNN used for the arrow classification

Capsule Network: We have used this network as it has proven to be effective in classifying overlapping characters on the MNIST [3] dataset. We set the learning

rate to 0.0005 and use the Adam Optimizer to train our model on all the single characters as well as on all the possible pairs of characters touching each other.

Spatial Transformer Network (STN) These are convolutional neural networks, containing one or several Spatial Transformer Modules. These modules try to make the network spatially invariant to its input data, in a computationally efficient manner, leading to more accurate object classification results. We have taken the architecture from [7]. All the input images are padded and resized to 32×32 so that they do not loose their original aspect ratio. We trained this network on images of all the 31 characters.

4.2 Results

We present results for individual components as well as the overall performance of the model.

Table 1. Accuracy of Individual Components for Text Extraction and Association

Component	Accuracy
Connector Detection	89.7%
CTPN	91.6%
Patch Association	95.1%
Clustering	95.6%
Zone mapping	96.4%

Table 2. Accuracy of Individual Components for Text Reading

Component	Accuracy
CCA	97.54%
Ranking	98.08%
CapsNet(Overlap)	66.11%
CapsNet(Non-Overlap)	89.59%
STN	95.06%
Sequence Reading	94.63%

Table 3. Cumulative accuracy for the complete framework

Component	Individual Accuracy	Cumulative Accuracy
Text Association	87.1%	87.1%
Text Reading	94.63%	82.3%

The results of Connector Detection is shown in Table 1. A total of 385 arrows were correctly localized out of 429 arrows present. The detection was performed on the sheets where the templates were removed. Majority of the false negatives occured as a result of probabilistic hough lines missing the entire line or most of the line, resulting in its removal during the arrow filtering stage.
The result of text patch detection using CTPN is shown in Table 1. It detected 392 text patches out of a total of 429 text patches correctly. It missed a few patches entirely and it resulted in a few false negatives in which it was generating a bounding box enclosing more than a single text patch inside it.

Out of the 392 text patches that the CTPN detected, 374 were correctly associated with the correct arrow, giving us the Patch Association accuracy as shown in Table 1.
And for the boxes which were associated with multiple arrows(false negative of CTPN enclosing more than a single text patch), we applied k-means clustering on the connented components present inside the CTPN boxes. It resulted in clusters of connented components belonging to the same text patch. Out of 23 such text patches which asked for clustering, k-menas clustering 22 of them correctly yielding an overall accuracy of 95.6% as shown in Table 1
We present the results of the text reading module in Table 2. We performed our experiments on 349 image patches. The accuracy of the CCA is calculated as the percentage of correct characters outputs in the total number of outputs. Ranking accuracy is calculated as a percentage of correct rankings done by the total number of images patches. The performance of the capsule network has been measured for two tasks (mentioned in the Table 2 above), one being the recognition of the overlapping characters and second, character level recognition in cases of non-overlapping characters. And at last the STN accuracy shows the character level accuracy which is better than the character level accuracy of the Capsule Network, justifying the reason why STN was used in the first place. Now the sequence level recognition's accuracy can be measured by measuring the ground-truth as well as the final predictions of the networks passing through both the correction modules, which is shown in the Table 2. The way we consider a prediction correct is if and only if all the characters in the predicted string matches with the ground-truth in the correct order.
The cumulative accuracy of the framework is provided in Table 3.

5 Conclusion

The proposed framework has given a detection accuracy of 87.1% for detection and 94.63% for reading. It manages to achieve high accuracy and is robust to different types of noise in arrow / cloud / text detection and character recognition. While it may be possible to train a deep system or model to learn this task in an end to end fashion given a very large set of cleanly annotated documents, but with the limited data at our disposal, incorporation of domain information was mandatory. As the entire pipeline is dedicated to a given layout, we plan to formulate an approach that is customizable with different layout types in future.

References

1. Agin, G.J.: Computer vision systems for industrial inspection and assembly. Computer (5), 11–20 (1980)
2. Badrinarayanan, V., Kendall, A., Cipolla, R.: Segnet: A deep convolutional encoder-decoder architecture for image segmentation. IEEE transactions on pattern analysis and machine intelligence **39**(12), 2481–2495 (2017)
3. Deng, L.: The mnist database of handwritten digit images for machine learning research [best of the web]. IEEE Signal Processing Magazine **29**(6), 141–142 (2012)

4. Glorot, X., Bengio, Y.: Understanding the difficulty of training deep feedforward neural networks. In: Proceedings of the thirteenth international conference on artificial intelligence and statistics. pp. 249–256 (2010)
5. Golnabi, H., Asadpour, A.: Design and application of industrial machine vision systems. Robotics and Computer-Integrated Manufacturing **23**(6), 630–637 (2007)
6. Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. arXiv preprint arXiv:1502.03167 (2015)
7. Jaderberg, M., Simonyan, K., Zisserman, A., et al.: Spatial transformer networks. In: Advances in neural information processing systems. pp. 2017–2025 (2015)
8. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980 (2014)
9. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: Advances in neural information processing systems. pp. 1097–1105 (2012)
10. Marinai, S.: Introduction to document analysis and recognition. In: Machine learning in document analysis and recognition, pp. 1–20. Springer (2008)
11. Ramakrishna, P., Hassan, E., Hebbalaguppe, R., Sharma, M., Gupta, G., Vig, L., Sharma, G., Shroff, G.: An ar inspection framework: Feasibility study with multiple ar devices. In: 2016 IEEE International Symposium on Mixed and Augmented Reality (ISMAR-Adjunct). pp. 221–226 (Sept 2016). https://doi.org/10.1109/ISMAR-Adjunct.2016.0080
12. Sabour, S., Frosst, N., Hinton, G.E.: Dynamic routing between capsules. In: Advances in Neural Information Processing Systems. pp. 3859–3869 (2017)
13. Seddati, O., Dupont, S., Mahmoudi, S.: Deepsketch: deep convolutional neural networks for sketch recognition and similarity search. In: Content-Based Multimedia Indexing (CBMI), 2015 13th International Workshop on. pp. 1–6. IEEE (2015)
14. Shimrat, M.: Algorithm 112: position of point relative to polygon. Communications of the ACM **5**(8), 434 (1962)
15. Tian, Z., Huang, W., He, T., He, P., Qiao, Y.: Detecting text in natural image with connectionist text proposal network. In: European conference on computer vision. pp. 56–72. Springer (2016)
16. Yoo, J.C., Han, T.H.: Fast normalized cross-correlation. Circuits Syst. Signal Process. **28**(6), 819–843 (Dec 2009). https://doi.org/10.1007/s00034-009-9130-7, `http://dx.doi.org/10.1007/s00034-009-9130-7`
17. Zeiler, M.D., Fergus, R.: Visualizing and understanding convolutional networks. In: European conference on computer vision. pp. 818–833. Springer (2014)